Algoritmos - Actividad Guiada 2

Nombre: Marc González Pastor
URL: https://colab.research.google.com/github/marcgpastor/03_MIAR_Actividades/blob/master/02_Algoritmos_AG2.ipynb
Repositorio GitHub: https://github.com/marcgpastor/03_MIAR_Actividades

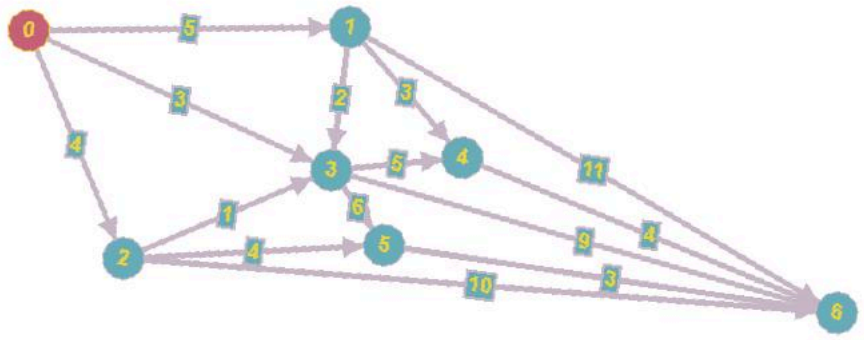
```
import itertools
import random
import re
import time
from typing import List, Optional, Sequence, Set, Tuple

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import sympy as sp
from sympy.plotting import plot3d
```

Programación Dinámica. Viaje por el río

- **Definición**: Es posible dividir el problema en subproblemas más pequeños, guardando las soluciones para ser utilizadas más adelante.
- **Características** que permiten identificar problemas aplicables:
  - Es posible almacenar soluciones de los subproblemas para ser utilizados más adelante
  - Debe verificar el principio de optimalidad de Bellman: "en una secuencia optima de decisiones, toda sub-secuencia también es óptima" (*)
  - La necesidad de guardar la información acerca de las soluciones parciales unido a la recursividad provoca la necesidad de preocuparnos por la complejidad espacial (cuantos recursos de espacio usaremos)

Problema

En un río hay **n** embarcaderos y debemos desplazarnos río abajo desde un embarcadero a otro. Cada embarcadero tiene precios diferentes para ir de un embarcadero a otro situado más abajo. Para ir del embarcadero i al j, puede ocurrir que sea más barato hacer un trasbordo por un embarcadero intermedio k. El problema consiste en determinar la combinación más barata.



- Consideramos una tabla TARIFAS(i,j) para almacenar todos los precios que nos ofrecen los embarcaderos.
- Si no es posible ir desde i a j daremos un valor alto para garantizar que ese trayecto no se va a elegir en la ruta óptima(modelado habitual para restricciones)

```
INF = float("inf")

tarifas = [
    [0, 5, 4, 3, INF, INF, INF],
    [INF, 0, INF, 2, 3, INF, 11],
    [INF, INF, 0, 1, INF, 4, 10],
    [INF, INF, INF, 0, 5, 6, 9],
    [INF, INF, INF, INF, 0, INF, 4],
    [INF, INF, INF, INF, INF, 0, 3],
    [INF, INF, INF, INF, INF, INF, 0]
]
```

```
def calcular_precios_ruta(tarifas: List[List[float]]) -> Tuple[List[List[float]], List[List[Optional[int]]]]:
    """Calcula las matrices de precios mínimos y nodos para reconstruir rutas.
```

```python
    precios[i][j] almacena el coste mínimo para ir del nodo i al nodo j (solo para i < j).
    ruta[i][j] almacena el nodo k que permite obtener ese coste mínimo en el último tramo k -> j.
    Si la mejor opción es ir directamente de i a j, entonces ruta[i][j] = i.

    Nota: este algoritmo rellena únicamente la parte superior de las matrices (i < j), asumiendo
    que el recorrido avanza hacia adelante (no hay ciclos ni retrocesos).

    Args:
        tarifas: Matriz cuadrada con los costes directos. Si no hay un camino directo, se debe usar
            un valor alto (ej. float('inf')).

    Returns:
        Una tupla (precios, ruta) donde:
        - precios: Matriz con los costes mínimos para ir de i a j.
        - ruta: Matriz para reconstruir la ruta óptima. Las posiciones no calculadas (i >= j) se dejan como None.
    """
    # Total de nodos
    n = len(tarifas)        # Al ser una matriz cuadrada, len(tarifas) == len(tarifas[0])

    # Inicialización de las matrices de precios y rutas
    precios: List[List[float]] = [[INF] * n for _ in range(n)]
    ruta: List[List[Optional[int]]] = [[None] * n for _ in range(n)]
    for i in range(n):
        precios[i][i] = 0.0
        ruta[i][i] = i

    # Construcción de precios y ruta para cada par (origen, destino)
    for i in range(n - 1):              # Origen
        for j in range(i + 1, n):       # Destino
            minimo = tarifas[i][j]
            ruta[i][j] = i              # Inicialmente, asumimos que la mejor ruta es la directa

            for k in range(i + 1, j):   # Nodo intermedio
                candidato = precios[i][k] + tarifas[k][j]
                if candidato < minimo:
                    minimo = candidato
                    ruta[i][j] = k      # Actualizamos el nodo intermedio

            precios[i][j] = minimo      # Guardamos el coste mínimo encontrado

    return precios, ruta
```

```python
def calcular_ruta(ruta: List[List[Optional[int]]], origen: int, destino: int) -> str:
    """Reconstruye la ruta óptima a partir de la matriz `ruta`.

    La matriz `ruta` se interpreta así:
      - ruta[i][j] = i si el mejor camino es directo i -> j.
      - ruta[i][j] = k (con i < k < j) si el mejor camino termina con k -> j
      y el tramo i -> k debe reconstruirse recursivamente.

    Args:
        ruta: Matriz devuelta por `calcular_precios_ruta`.
        origen: Nodo de inicio.
        destino: Nodo de destino.

    Returns:
        Ruta completa como cadena con nodos separados por comas (p. ej. "0,2,4,6").
    """
    k = ruta[origen][destino]
    if k is None:
        raise ValueError(f"No hay información de ruta para ({origen}, {destino}).")

    if k == origen:
        return f"{origen},{destino}"

    return f"{calcular_ruta(ruta, origen, k)},{destino}"
```

```python
precios, ruta = calcular_precios_ruta(tarifas)

origen = 0
destino = 6
print("Coste mínimo 0→6:", precios[origen][destino])
print("Ruta 0→6:", calcular_ruta(ruta, origen, destino))

print("\nPRECIOS")
for fila in precios:
    print(fila)

print("\nRUTA")
for fila in ruta:
    print(fila)
```

```
Coste mínimo 0→6: 11
Ruta 0→6: 0,2,5,6

PRECIOS
[0.0, 5, 4, 3, 8, 8, 11]
```

```
[inf, 0.0, inf, 2, 3, 8, 7]
[inf, inf, 0.0, 1, 6, 4, 7]
[inf, inf, inf, 0.0, 5, 6, 9]
[inf, inf, inf, inf, 0.0, inf, 4]
[inf, inf, inf, inf, inf, 0.0, 3]
[inf, inf, inf, inf, inf, inf, 0.0]

RUTA
[0, 0, 0, 0, 1, 2, 5]
[None, 1, 1, 1, 1, 3, 4]
[None, None, 2, 2, 3, 2, 5]
[None, None, None, 3, 3, 3, 3]
[None, None, None, None, 4, 4, 4]
[None, None, None, None, None, 5, 5]
[None, None, None, None, None, None, 6]
```

Problema de Asignación de tarea

```
# Asignación de tareas — Ramificación y Poda
# costes[i][j]: coste de asignar un elemento al otro.
# La tupla solución S se interpreta como: S[i] = asignación elegida para el índice i (agente i asignado a la tarea S[i]).

INF = float("inf")

costes = [
    [11, 12, 18, 40],
    [14, 15, 13, 22],
    [11, 17, 19, 23],
    [17, 14, 20, 28],
]

objetivo = "max"
```

```
def coste_acumulado(solucion: Sequence[int], costes: List[List[float]]) -> float:
    """Calcula el coste acumulado de una solución (parcial o completa).

    Convención:
        solucion[i] = tarea asignada al agente i
        costes[i][j] = coste de asignar al agente i la tarea j

    Args:
        solucion: Asignación parcial o completa.
        costes: Matriz cuadrada de costes (n x n).

    Returns:
        Coste total acumulado para las asignaciones ya decididas.
    """
    return sum(costes[agente][tarea] for agente, tarea in enumerate(solucion))


def tareas_asignadas(solucion: Sequence[int]) -> Set[int]:
    """Devuelve el conjunto de tareas ya asignadas en la solución parcial.

    Args:
        solucion: Asignación parcial (tupla de tareas asignadas a agentes 0, ..., n-1).

    Returns:
        Conjunto de índices de tareas que ya han sido asignadas.
    """
    return set(solucion)


def tareas_disponibles(solucion: Sequence[int], n: int) -> Set[int]:
    """Devuelve el conjunto de tareas que aún no han sido asignadas.

    Args:
        solucion: Asignación parcial.
        n: Número total de tareas/agentes (tamaño del problema).

    Returns:
        Conjunto de índices de tareas disponibles para asignar.
    """
    asignadas = tareas_asignadas(solucion)
    return set(range(n)) - asignadas


def cota_inferior(solucion: Sequence[int], costes: List[List[float]]) -> float:
    """Calcula una cota inferior (optimista) del coste total para una solución parcial.

    Parte 1 (coste ya fijado):
        Suma los costes de las asignaciones ya decididas.

    Parte 2 (estimación optimista para completar):
        Para cada tarea j aún no asignada (tareas_disponibles),
        añade el mínimo (o máximo) coste posible en esa columna considerando
        solo los agentes restantes (i desde len(solucion) hasta n).
```

```
    Args:
        solucion: Asignación parcial (longitud m). `solucion[i]` es el índice asignado en i.
        costes: Matriz cuadrada de costes (n x n).

    Returns:
        Un valor que actúa como cota inferior del coste de cualquier completación.
    """
    m = len(solucion)
    n = len(costes)

    total = coste_acumulado(solucion, costes)

    for j in tareas_disponibles(solucion, n):
        total += min(costes[i][j] for i in range(m, n))

    return total


def cota_superior(solucion: Sequence[int], costes: List[List[float]]) -> float:
    """Calcula una cota superior (pesimista) del coste total para una solución parcial.

    Misma idea que `cota_inferior`, pero usando máximos en la estimación.

    Args:
        solucion: Asignación parcial.
        costes: Matriz cuadrada de costes (n x n).

    Returns:
        Un valor que actúa como cota superior.
    """
    m = len(solucion)
    n = len(costes)

    total = coste_acumulado(solucion, costes)

    for j in tareas_disponibles(solucion, n):
        total += max(costes[i][j] for i in range(m, n))

    return total


def crear_hijos(solucion: Tuple[int, ...], n: int) -> List[Tuple[int, ...]]:
    """Amplía la solución con el resto de posibilidades.

    Dado un conjunto de nodos (tupla de asignaciones ya decididas), se añaden las posibles asignaciones para el siguiente agente (índice

    Args:
        solucion: Solución parcial actual (tupla de asignaciones).
        n: Número total de tareas/agentes.

    Returns:
        Lista de nodos hijos (tuplas), cada uno con una nueva asignación añadida.
    """

    hijos = []
    for tarea in sorted(tareas_disponibles(solucion, n)):
        hijos.append(solucion + (tarea,))

    return hijos
```

```
def ramificacion_y_poda(costes, objetivo="min", tiempo_limite_s=None) -> tuple[tuple[int] | tuple[int, ...], float, int]:
    """Resuelve el problema de asignación mediante Ramificación y Poda (Branch & Bound).

    Convención:
        - La solución se representa como una tupla `s` donde `s[i]` es la tarea asignada al agente i.
        - La matriz `costes` se interpreta como `costes[agente][tarea]`.

    Estrategia:
        - Se mantiene una lista de nodos pendientes, donde cada nodo es un diccionario:
            {"s": solucion_parcial, "cota": cota_asociada}
        - En cada iteración se selecciona el nodo más prometedor:
            * Minimización: el de menor cota (cota inferior).
            * Maximización: el de mayor cota (cota superior).
        - Se ramifica generando hijos (añadiendo una tarea al siguiente agente).
        - Si aparece una solución completa, se actualiza la mejor solución conocida.
        - Se poda descartando nodos cuya cota ya no permite mejorar la mejor solución actual.

    Args:
        costes: Matriz cuadrada de costes (n x n).
        objetivo: "min" para minimizar el coste total, "max" para maximizarlo.
        tiempo_limite_s: criterio de parada, opcional.

    Returns:
        Una tupla (mejor_solucion, mejor_valor, iteraciones) donde:
        - mejor_solucion: tupla con la asignación óptima encontrada.
        - mejor_valor: coste total asociado a `mejor_solucion` (según el objetivo).
```

```
        - iteraciones: número de iteraciones del bucle principal (nodos extraídos/expandido).
    """
    dimension = len(costes)

    mejor_solucion = tuple(range(dimension))
    mejor_valor = coste_acumulado(mejor_solucion, costes)

    if objetivo == "max":
        cota = cota_superior
        seleccionar = max
        mejora = lambda v: v > mejor_valor
        no_puede_mejorar = lambda c: c <= mejor_valor
        filtrar_pendientes = lambda nodos: [x for x in nodos if x["cota"] > mejor_valor]
    else:
        cota = cota_inferior
        seleccionar = min
        mejora = lambda v: v < mejor_valor
        no_puede_mejorar = lambda c: c >= mejor_valor
        filtrar_pendientes = lambda nodos: [x for x in nodos if x["cota"] < mejor_valor]

    t_ini = time.perf_counter()
    nodos = [{"s": (), "cota": cota((), costes)}]
    iteraciones = 0

    while nodos:
        if tiempo_limite_s is not None and (time.perf_counter() - t_ini) > tiempo_limite_s:
            raise TimeoutError("Límite de tiempo excedido en Ramificación y Poda")

        iteraciones += 1
        nodo_prometedor = seleccionar(nodos, key=lambda x: x["cota"])

        if no_puede_mejorar(nodo_prometedor["cota"]):
            break

        solucion_parcial = nodo_prometedor["s"]
        nodos = [x for x in nodos if x["s"] != solucion_parcial]

        for hijo in crear_hijos(solucion_parcial, dimension):
            cota_hijo = cota(hijo, costes)

            if no_puede_mejorar(cota_hijo):
                continue

            if len(hijo) == dimension:
                valor_hijo = coste_acumulado(hijo, costes)
                if mejora(valor_hijo):
                    mejor_valor = valor_hijo
                    mejor_solucion = hijo
                    nodos = filtrar_pendientes(nodos)
                continue

            nodos.append({"s": hijo, "cota": cota_hijo})

    return mejor_solucion, mejor_valor, iteraciones
```

```
def fuerza_bruta(costes, objetivo="min", tiempo_limite_s=None) -> tuple[tuple[int, ...] | None, float, int]:
    """Resuelve asignación probando todas las permutaciones (búsqueda exhaustiva).

    Convención:
        solucion[i] = tarea asignada al agente i
        costes[i][j] = coste de asignar al agente i la tarea j

    Args:
        costes: Matriz cuadrada (n x n).
        objetivo: "min" para minimizar, "max" para maximizar.
        tiempo_limite_s: criterio de parada, opcional.

    Returns:
        (mejor_solucion, mejor_valor, iteraciones)
    """
    n = len(costes)
    mejor_solucion = None
    mejor_valor = -INF if objetivo == "max" else INF
    iteraciones = 0

    t0 = time.perf_counter()

    for perm in itertools.permutations(range(n)):
        iteraciones += 1

        if tiempo_limite_s is not None and (time.perf_counter() - t0) > tiempo_limite_s:
            raise TimeoutError("Límite de tiempo excedido en Fuerza Bruta")

        valor = coste_acumulado(perm, costes)

        if objetivo == "max":
            if valor > mejor_valor:
```

```
                mejor_valor = valor
                mejor_solucion = perm
        else:
            if valor < mejor_valor:
                mejor_valor = valor
                mejor_solucion = perm

    return mejor_solucion, mejor_valor, iteraciones
```

```
t0_bb = time.perf_counter()
sol_bb, val_bb, iter_bb = ramificacion_y_poda(costes, objetivo=objetivo)
t1_bb = time.perf_counter()
print(
    "Resultado (Ramificación y Poda)\n"
    f"- Objetivo     : {'minimizar' if objetivo == 'min' else 'maximizar'}\n"
    f"- Dimensión    : {len(costes)}\n"
    f"- Solución     : {sol_bb}\n"
    f"- Valor total  : {val_bb}\n"
    f"- Iteraciones  : {iter_bb}\n"
    f"- Tiempo (s)   : {t1_bb - t0_bb:.4f}"
)
```

```
Resultado (Ramificación y Poda)
- Objetivo     : maximizar
- Dimensión    : 4
- Solución     : (3, 1, 2, 0)
- Valor total  : 91
- Iteraciones  : 5
- Tiempo (s)   : 0.0001
```

```
t0_fb = time.perf_counter()
sol_fb, val_fb, iter_fb = fuerza_bruta(costes, objetivo=objetivo)
t1_fb = time.perf_counter()

print(
    "Resultado (Fuerza Bruta)\n"
    f"- Objetivo       : {'minimizar' if objetivo == 'min' else 'maximizar'}\n"
    f"- Dimensión      : {len(costes)}\n"
    f"- Solución       : {sol_fb}\n"
    f"- Valor total    : {val_fb}\n"
    f"- Evaluaciones   : {iter_fb}\n"
    f"- Tiempo (s)     : {t1_fb - t0_fb:.4f}"
)
```

```
Resultado (Fuerza Bruta)
- Objetivo       : maximizar
- Dimensión      : 4
- Solución       : (3, 0, 1, 2)
- Valor total    : 91
- Evaluaciones   : 24
- Tiempo (s)     : 0.0000
```

```
def generar_matriz_costes(n, low=1, high=100, seed=None) -> list[list[int]]:
    """Genera una matriz n x n de costes enteros aleatorios.

    Args:
        n: dimensión.
        low: valor mínimo.
        high: valor máximo.
        seed: semilla.

    Returns:
        list: Matriz de costes.

    """
    rng = random.Random(seed)
    return [[rng.randint(low, high) for _ in range(n)] for _ in range(n)]

def ejecutar_ramificacion_y_poda(costes, objetivo="min") -> dict[str, str | int]:
    """Ejecuta Ramificación y Poda y devuelve métricas.

    Args:
        costes: Matriz cuadrada (n x n).
        objetivo: "min" para minimizar, "max" para maximizar.

    Returns:
        Diccionario con los resultados.
    """
    t0_bb = time.perf_counter()
    solucion, valor, iteraciones = ramificacion_y_poda(costes, objetivo=objetivo)
    t1_bb = time.perf_counter()

    return {
        "algoritmo": "ramificacion_y_poda",
        "tiempo_s": t1_bb - t0_bb,
        "pasos": iteraciones,
        "valor": valor,
        "solucion": str(solucion),
```

```
    }

def ejecutar_fuerza_bruta(costes, objetivo="min") -> dict[str, str | int]:
    """Ejecuta Fuerza Bruta y devuelve métricas.

    Args:
        costes: Matriz cuadrada (n x n).
        objetivo: "min" para minimizar, "max" para maximizar.

    Returns:
        Diccionario con los resultados.
    """
    t0_fb = time.perf_counter()
    solucion, valor, evaluaciones = fuerza_bruta(costes, objetivo=objetivo)
    t1_fb = time.perf_counter()
    return {
        "algoritmo": "fuerza_bruta",
        "tiempo_s": t1_fb - t0_fb,
        "pasos": evaluaciones,
        "valor": valor,
        "solucion": str(solucion),
    }

def benchmark_asignacion(
    n_min=2,
    n_max=8,
    objetivo="min",
    seed_base=0,
    repeticiones=1,
    low=1,
    high=100,
    csv_path=None,
) -> pd.DataFrame:
    """Benchmark de asignación (Fuerza Bruta vs. Ramificación y Poda).

    Para cada tamaño n y repetición:
      - genera una matriz aleatoria con una semilla reproducible
      - ejecuta ambos algoritmos sobre la misma matriz
      - devuelve un DataFrame con los resultados

    Si `csv_path` no es None, guarda el DataFrame en CSV.

    Args:
        n_min:
        n_max:
        objetivo:
        seed_base:
        repeticiones:
        low:
        high:
        csv_path:

    Returns:

    """
    filas = []

    for n in range(n_min, n_max + 1):
        for r in range(repeticiones):
            seed = seed_base + 1000 * n + r
            costes = generar_matriz_costes(n, low=low, high=high, seed=seed)

            out_bb = ejecutar_ramificacion_y_poda(costes, objetivo=objetivo)
            out_fb = ejecutar_fuerza_bruta(costes, objetivo=objetivo)

            if out_bb["valor"] != out_fb["valor"]:
                print(f"AVISO: n={n}, seed={seed}, BB={out_bb['valor']} vs FB={out_fb['valor']}")

            for out in (out_bb, out_fb):
                filas.append({
                    "n": n,
                    "objetivo": objetivo,
                    "seed": seed,
                    "repetición": r,
                    **out,
                })

    df = pd.DataFrame(filas)

    if csv_path is not None:
        df.to_csv(csv_path, index=False, encoding="utf-8")

    return df
```

```
df = benchmark_asignacion(
    n_min=5,
    n_max=10,
```

```
    objetivo="max",
    seed_base=100,
    repeticiones=3
)

df
```

	n	objetivo	seed	repetición	algoritmo	tiempo_s	pasos	valor	solucion
0	5	max	5100	0	ramificacion_y_poda	0.000055	5	427	(1, 2, 4, 3, 0)
1	5	max	5100	0	fuerza_bruta	0.000045	120	427	(1, 2, 4, 3, 0)
2	5	max	5101	1	ramificacion_y_poda	0.000038	5	393	(1, 4, 0, 2, 3)
3	5	max	5101	1	fuerza_bruta	0.000040	120	393	(1, 4, 0, 2, 3)
4	5	max	5102	2	ramificacion_y_poda	0.000067	10	356	(1, 3, 2, 0, 4)
5	5	max	5102	2	fuerza_bruta	0.000040	120	356	(1, 3, 2, 0, 4)
6	6	max	6100	0	ramificacion_y_poda	0.000113	15	473	(0, 4, 5, 3, 2, 1)
7	6	max	6100	0	fuerza_bruta	0.000244	720	473	(0, 4, 5, 3, 2, 1)
8	6	max	6101	1	ramificacion_y_poda	0.000163	25	482	(0, 1, 2, 3, 5, 4)
9	6	max	6101	1	fuerza_bruta	0.000254	720	482	(0, 1, 2, 3, 5, 4)
10	6	max	6102	2	ramificacion_y_poda	0.000057	7	518	(0, 2, 4, 1, 3, 5)
11	6	max	6102	2	fuerza_bruta	0.000291	720	518	(0, 2, 4, 1, 3, 5)
12	7	max	7100	0	ramificacion_y_poda	0.000128	12	543	(1, 0, 6, 5, 2, 4, 3)
13	7	max	7100	0	fuerza_bruta	0.001827	5040	543	(1, 0, 6, 5, 2, 4, 3)
14	7	max	7101	1	ramificacion_y_poda	0.000193	18	600	(4, 2, 0, 6, 1, 3, 5)
15	7	max	7101	1	fuerza_bruta	0.002481	5040	600	(4, 2, 0, 6, 1, 3, 5)
16	7	max	7102	2	ramificacion_y_poda	0.000201	17	587	(1, 6, 3, 2, 5, 0, 4)
17	7	max	7102	2	fuerza_bruta	0.001818	5040	587	(1, 6, 3, 2, 5, 0, 4)
18	8	max	8100	0	ramificacion_y_poda	0.000117	9	667	(1, 6, 3, 5, 4, 0, 7, 2)
19	8	max	8100	0	fuerza_bruta	0.017511	40320	667	(1, 6, 3, 5, 4, 0, 7, 2)
20	8	max	8101	1	ramificacion_y_poda	0.000508	33	718	(0, 7, 5, 3, 2, 6, 1, 4)
21	8	max	8101	1	fuerza_bruta	0.016415	40320	718	(0, 7, 5, 3, 2, 6, 1, 4)
22	8	max	8102	2	ramificacion_y_poda	0.003650	147	573	(3, 5, 0, 4, 2, 7, 6, 1)
23	8	max	8102	2	fuerza_bruta	0.015854	40320	573	(3, 5, 0, 4, 2, 7, 6, 1)
24	9	max	9100	0	ramificacion_y_poda	0.000204	13	810	(7, 0, 4, 2, 6, 3, 1, 5, 8)
25	9	max	9100	0	fuerza_bruta	0.157712	362880	810	(7, 0, 4, 2, 6, 3, 1, 5, 8)
26	9	max	9101	1	ramificacion_y_poda	0.000404	20	797	(0, 8, 2, 1, 7, 4, 5, 3, 6)
27	9	max	9101	1	fuerza_bruta	0.163317	362880	797	(0, 8, 2, 1, 7, 4, 5, 3, 6)
28	9	max	9102	2	ramificacion_y_poda	0.008274	235	716	(7, 6, 0, 4, 8, 2, 1, 3, 5)
29	9	max	9102	2	fuerza_bruta	0.252145	362880	716	(7, 6, 0, 4, 8, 2, 1, 3, 5)
30	10	max	10100	0	ramificacion_y_poda	0.006348	161	869	(9, 5, 3, 7, 0, 2, 4, 8, 1, 6)
31	10	max	10100	0	fuerza_bruta	1.687905	3628800	869	(9, 5, 3, 7, 0, 2, 4, 8, 1, 6)
32	10	max	10101	1	ramificacion_y_poda	0.035930	545	876	(4, 8, 7, 9, 2, 6, 0, 3, 5, 1)
33	10	max	10101	1	fuerza_bruta	1.678857	3628800	876	(4, 8, 7, 9, 2, 6, 0, 3, 5, 1)
34	10	max	10102	2	ramificacion_y_poda	0.039103	602	838	(5, 3, 7, 2, 0, 9, 6, 4, 8, 1)
35	10	max	10102	2	fuerza_bruta	1.663135	3628800	838	(5, 3, 7, 1, 2, 0, 8, 4, 9, 6)

```
resumen = (
    df.groupby(["n", "algoritmo"], as_index=False)
      .agg(tiempo_mediana_s=("tiempo_s", "median"),
           pasos_mediana=("pasos", "median"),
           valor_mediana=("valor", "median"))
)

resumen
```

	n	algoritmo	tiempo_mediana_s	pasos_mediana	valor_mediana
0	5	fuerza_bruta	0.000040	120.0	393.0
1	5	ramificacion_y_poda	0.000055	5.0	393.0
2	6	fuerza_bruta	0.000254	720.0	482.0
3	6	ramificacion_y_poda	0.000113	15.0	482.0
4	7	fuerza_bruta	0.001827	5040.0	587.0
5	7	ramificacion_y_poda	0.000193	17.0	587.0
6	8	fuerza_bruta	0.016415	40320.0	667.0
7	8	ramificacion_y_poda	0.000508	33.0	667.0
8	9	fuerza_bruta	0.163317	362880.0	797.0
9	9	ramificacion_y_poda	0.000404	20.0	797.0
10	10	fuerza_bruta	1.678857	3628800.0	869.0
11	10	ramificacion_y_poda	0.035930	545.0	869.0

```
def ejecutar_bnb_con_timeout(costes, objetivo="min", timeout_s=5.0):
    t0 = time.perf_counter()
    try:
        sol, val, it = ramificacion_y_poda(costes, objetivo=objetivo, tiempo_limite_s=timeout_s)
        t1 = time.perf_counter()
        return {"n": len(costes), "timeout": False, "tiempo_s": t1 - t0, "iteraciones": it, "valor": val}
    except TimeoutError:
        t1 = time.perf_counter()
        return {"n": len(costes), "timeout": True, "tiempo_s": t1 - t0, "iteraciones": None, "valor": None}

def benchmark_bnb_solo(n_min=11, n_max=18, objetivo="min", repeticiones=3, seed_base=3000, timeout_s=5.0):
    filas = []
    for n in range(n_min, n_max + 1):
        for r in range(repeticiones):
            seed = seed_base + 10 * n + r
            costes = generar_matriz_costes(n, seed=seed)
            filas.append(ejecutar_bnb_con_timeout(costes, objetivo=objetivo, timeout_s=timeout_s))
    return pd.DataFrame(filas)

timeout = 5
df_bnb = benchmark_bnb_solo(n_min=11, n_max=18, objetivo=objetivo, repeticiones=3, timeout_s=timeout)
df_bnb
```

	n	timeout	tiempo_s	iteraciones	valor
0	11	False	0.005157	94.0	950.0
1	11	False	0.101931	782.0	925.0
2	11	False	0.008297	158.0	956.0
3	12	False	0.004558	97.0	1020.0
4	12	False	0.000833	28.0	1076.0
5	12	False	0.006683	131.0	1074.0
6	13	False	0.001653	49.0	1231.0
7	13	False	0.019704	233.0	1132.0
8	13	False	0.005067	94.0	1172.0
9	14	False	1.477376	2815.0	1258.0
10	14	False	0.520419	1603.0	1250.0
11	14	False	0.154387	850.0	1293.0
12	15	False	0.031087	322.0	1363.0
13	15	False	0.086889	510.0	1366.0
14	15	False	0.611481	1683.0	1346.0
15	16	False	3.338431	3464.0	1472.0
16	16	False	0.005135	77.0	1479.0
17	16	False	0.054137	415.0	1479.0
18	17	True	5.000341	NaN	NaN
19	17	False	1.448889	1980.0	1595.0
20	17	False	1.160559	2025.0	1600.0
21	18	True	5.003462	NaN	NaN
22	18	True	5.000907	NaN	NaN
23	18	False	3.899247	3411.0	1707.0

```
res_bnb = (
    df_bnb.assign(tiempo_ok=lambda d: d["tiempo_s"].where(~d["timeout"]))
          .groupby("n", as_index=False)
          .agg(
              ejecuciones=("timeout", "count"),
              timeouts=("timeout", "sum"),
              tiempo_mediana_s=("tiempo_ok", "median"),
              tiempo_max_s=("tiempo_ok", "max"),
          )
)

n_critico = res_bnb.loc[res_bnb["timeouts"] > 0, "n"].min()
print(f"Primer n con algún timeout (límite {timeout}s): {n_critico}")
```

```
Primer n con algún timeout (límite 5s): 17
```

Análisis para mejorar nota

Experimento

Se generan matrices aleatorias $n \times n$ para $n = 5, 6, \ldots, 10$ (con varias repeticiones) y se ejecutan:

- Fuerza bruta
- Ramificación y poda (B&B)

Los resultados agregados (medianas) se recogen en la tabla `resumen`.

---

1) Complejidad de fuerza bruta

Fuerza bruta explora todas las permutaciones de tareas, por tanto evalúa $n!$ asignaciones. Cada evaluación calcula el valor total en $O(n)$.

$$T_{\mathrm{FB}}(n) = O(n \cdot n!), \qquad S_{\mathrm{FB}}(n) = O(n).$$

En particular:

$$9! = 362\,880, \qquad 10! = 3\,628\,800.$$

---

2) ¿A partir de qué dimensión fuerza bruta deja de ser una opción?

Como $(n+1)! = (n+1)\,n!$, pasar de n a $n+1$ multiplica el trabajo aproximadamente por $(n+1)$. Si llamamos t_{10} al tiempo mediano observado en `resumen` para fuerza bruta en $n = 10$, entonces:

$$t_{11} \approx 11\,t_{10}, \qquad t_{12} \approx 12 \cdot 11\,t_{10}.$$

Con esta escalada factorial y los tiempos medidos en `resumen`, a partir de $n \approx 11$ (y con claridad en $n = 12$) fuerza bruta deja de ser práctica para comparar de forma sistemática.

---

3) ¿Existe un n a partir del cual B&B deja de ser una opción válida?

No hay un umbral fijo. El peor caso sigue siendo exponencial/factorial y depende de cuánto permitan podar las cotas en cada matriz.

Para objetivarlo, se ejecuta B&B en tamaños mayores ($n = 11, \ldots$) con un límite L de tiempo por instancia y se resume en `res_bnb`. El primer tamaño con algún `timeout` (para este L y estas semillas) es:

$$n_{\text{crítico}} = 17.$$

Esto marca el rango en el que, bajo un presupuesto de tiempo fijo, B&B puede dejar de ser una opción válida en algunas instancias.

Descenso del gradiente

```
class VectorSimbolico:
    """Vector de variables simbólicas de creación perezosa.

    Permite escribir expresiones del tipo x[0], x[1], ... sin fijar n por adelantado.
    Cada vez que se accede a x[i], se crea (si no existe) el símbolo sympy `x{i}`.

    Args:
        prefijo: Prefijo para el nombre de las variables (por defecto "x").
    """

    def __init__(self, prefijo: str = "x"):
        self.prefijo = prefijo
        self._cache = {}

    def __getitem__(self, i: int):
        """Devuelve la variable simbólica asociada al índice i.

        Args:
            i: Índice de la variable.

        Returns:
            Símbolo de SymPy con nombre f"{prefijo}{i}" (por ejemplo, x0, x1, ...).
        """
        if i not in self._cache:
```

```
            self._cache[i] = sp.Symbol(f"{self.prefijo}{i}", real=True)
        return self._cache[i]


def construir_f_y_gradiente(expr):
    """Construye f(X) y df(X) (gradiente) a partir de una expresión simbólica.

    La función detecta automáticamente las variables utilizadas en `expr` mediante
    `expr.free_symbols`, calcula el gradiente simbólico y crea funciones numéricas
    evaluables usando `sympy.lambdify`.

    Convención:
        - Las variables se ordenan por nombre (x0, x1, x2, ...) para fijar el orden
          esperado en el vector X.
        - La función `f` devuelve un escalar.
        - La función `df` devuelve una lista con el gradiente en el mismo orden.

    Args:
        expr: Expresión simbólica de SymPy (por ejemplo, x[0]**2 + x[1]**2).

    Returns:
        (f, df) donde:
        - f: función numérica f(X) que evalúa la expresión.
        - df: función numérica df(X) que evalúa el gradiente.
    """

    def clave_orden(simbolo):
        m = re.match(r"([A-Za-z_]+)(\d+)$", simbolo.name)
        if m:
            return (m.group(1), int(m.group(2)))
        return (simbolo.name, 0)

    def safe_exp(t, clip=50.0):
        # Evita overflow: exp(50) sigue siendo finita y grande, pero no inf.
        return np.exp(np.clip(t, -clip, clip))

    variables = tuple(sorted(expr.free_symbols, key=clave_orden))
    grad_expr = [sp.diff(expr, v) for v in variables]

    modules = [{"exp": safe_exp}, "numpy"]
    f_num = sp.lambdify(variables, expr, modules=modules)
    grad_num = sp.lambdify(variables, grad_expr, modules=modules)

    f = lambda X: f_num(*X)
    df = lambda X: list(grad_num(*X))
    return f, df
```

Vamos a minimizar la función paraboloide:

$$f(x, y) = x^2 + y^2.$$

Su gradiente es:

$$\nabla f(x, y) = (2x, 2y).$$

Aplicamos descenso del gradiente con paso fijo α:

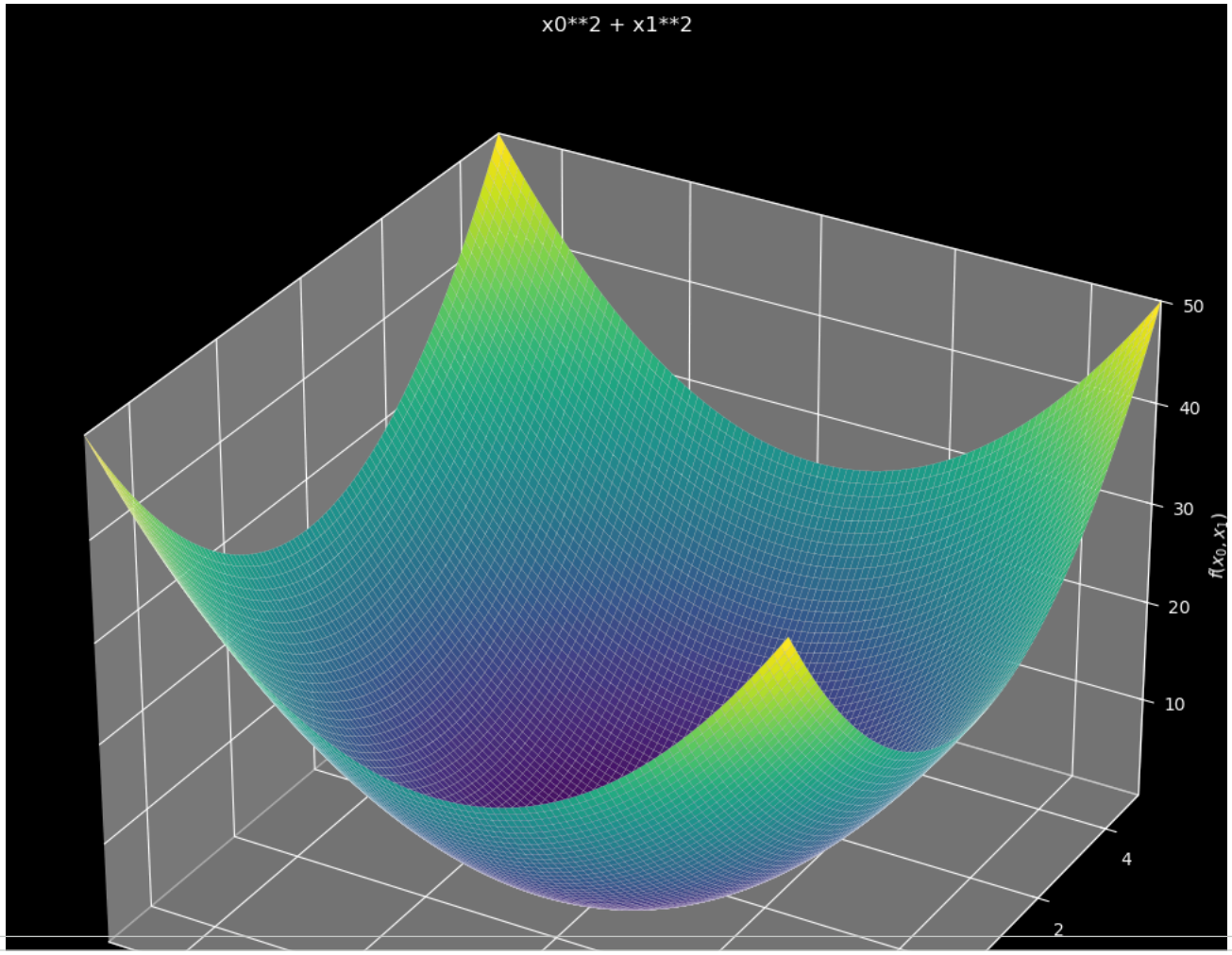
$$\begin{pmatrix} x_{k+1} \\ y_{k+1} \end{pmatrix} = \begin{pmatrix} x_k \\ y_k \end{pmatrix} - \alpha \nabla f(x_k, y_k).$$

El mínimo global es $(0, 0)$.

```
x = VectorSimbolico("x")

expr = x[0]**2 + x[1]**2
f, df = construir_f_y_gradiente(expr)

df([1, 2])
```

```
[2, 4]
```

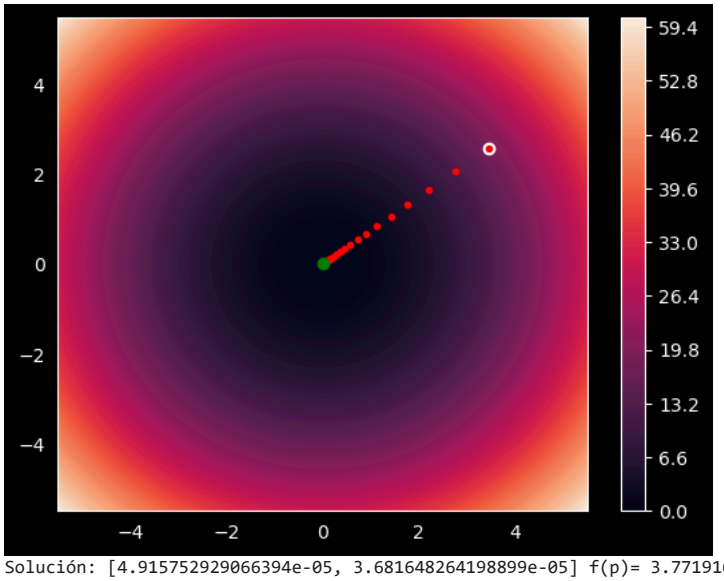
```
plot3d(
    expr,
    (x[0], -5, 5),
    (x[1], -5, 5),
    title=str(expr),
    size=(10, 10),
)
```




```
def ejecutar_descenso_gradiente(f, df, p0, tasa_aprendizaje=0.1, n_iter=50):
    """Ejecuta descenso del gradiente desde p0.

    Args:
        f: Función objetivo f(X) -> escalar.
        df: Gradiente df(X) -> lista/array.
        p0: Punto inicial (lista/tupla/np.array).
        tasa_aprendizaje: Step size fijo.
        n_iter: Número de iteraciones.

    Returns:
        (p_final, f(p_final), trayectoria) donde trayectoria es un array (n_iter+1, d).
    """
    p = np.asarray(p0, dtype=float)
    trayectoria = [p.copy()]

    for _ in range(n_iter):
        grad = np.asarray(df(p), dtype=float)
        p = p - tasa_aprendizaje * grad
        trayectoria.append(p.copy())

    return p, float(f(p)), np.vstack(trayectoria)


def dibujar_mapa_niveles(f, rango=5.5, resolucion=100):
    """Dibuja el mapa de niveles y devuelve (xs, ys, Z)."""
    xs = np.linspace(-rango, rango, resolucion)
    ys = np.linspace(-rango, rango, resolucion)
    X, Y = np.meshgrid(xs, ys)
    Z = f([X, Y])

    plt.contourf(xs, ys, Z, levels=resolucion)
    plt.colorbar()
    return xs, ys, Z
```

```
rng = random.Random(0)

dibujar_mapa_niveles(f, rango=5.5, resolucion=100)

p0 = [rng.uniform(-5, 5), rng.uniform(-5, 5)]
plt.plot(p0[0], p0[1], "o", c="white")

p_final, f_final, trayectoria = ejecutar_descenso_gradiente(
    f,
    df,
    p0,
    tasa_aprendizaje=0.1,
```

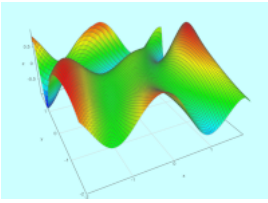
```
    n_iter=50,
)

plt.plot(trayectoria[:, 0], trayectoria[:, 1], "o", c="red", markersize=3)
plt.plot(p_final[0], p_final[1], "o", c="green")
plt.show()

print("Solución:", p_final.tolist(), "f(p)=", f_final)
```




```
Solución: [4.915752929066394e-05, 3.681648264198899e-05] f(p)= 3.77191608009036e-09
```

¿Te atreves a optimizar la función (no convexa)?

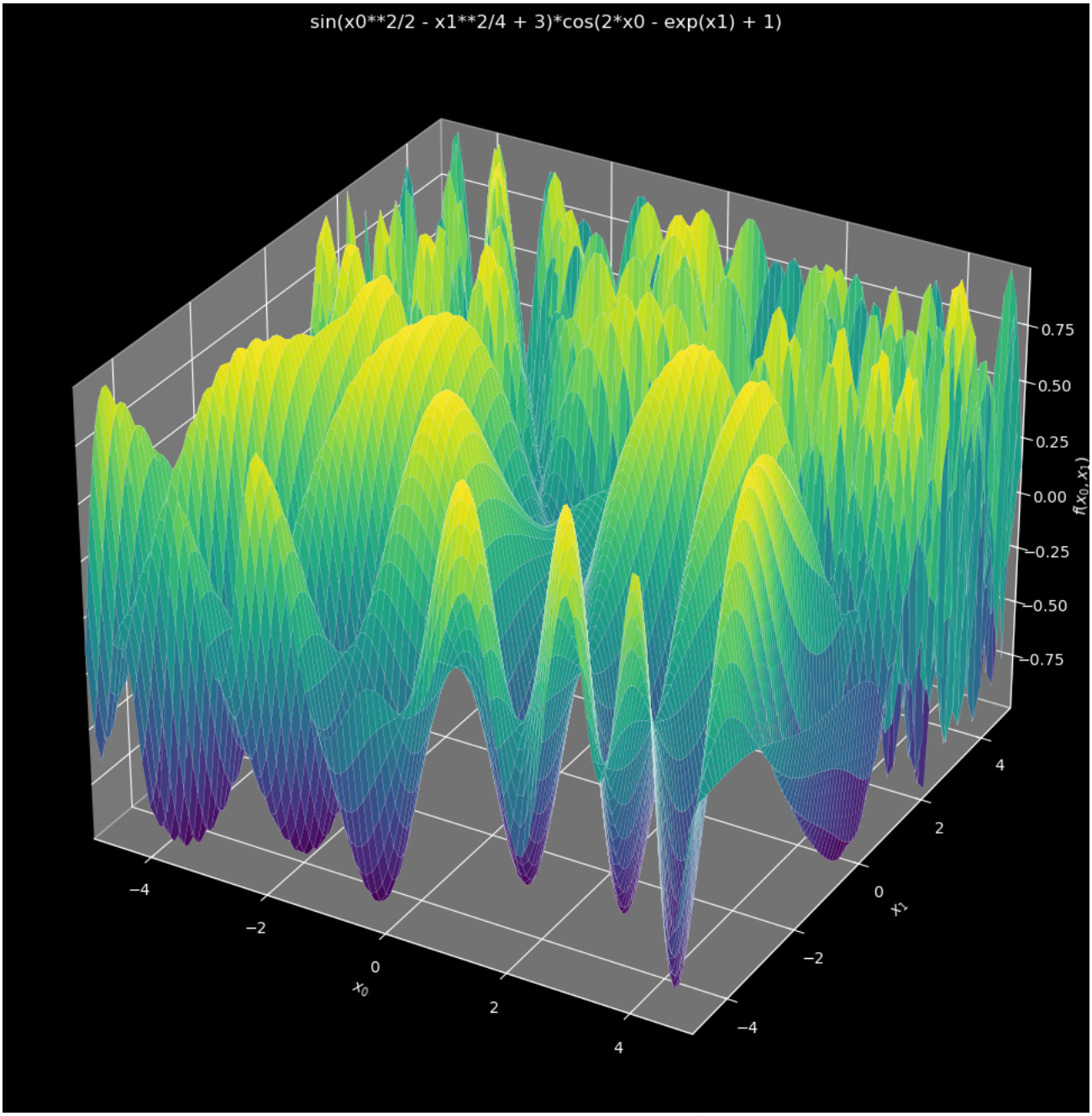
$$f(x,y) = \sin\left(\tfrac{1}{2}x^2 - \tfrac{1}{4}y^2 + 3\right) \cdot \cos(2x + 1 - e^y)\,.$$



```
x = VectorSimbolico("x")

expr = sp.sin(sp.Rational(1, 2) * x[0]**2 - sp.Rational(1, 4) * x[1]**2 + 3) * sp.cos(2 * x[0] + 1 - sp.exp(x[1]))
f, df = construir_f_y_gradiente(expr)

df([1, 2])
```

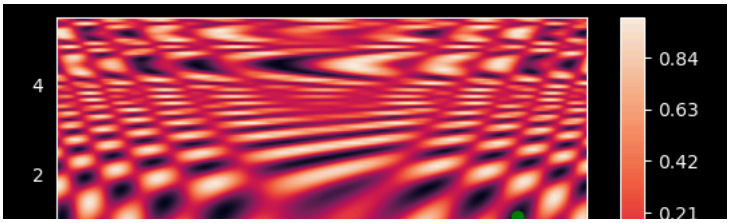
```
[np.float64(-0.8803745550791662), np.float64(3.9384502467765357)]
```

```
plot3d(
    expr,
    (x[0], -5, 5),
    (x[1], -5, 5),
    title=str(expr),
    size=(10, 10),
)
```




```
<sympy.plotting.backends.matplotlibbackend.matplotlib.MatplotlibBackend at 0x797fe8a64190>
```

```
rng = random.Random(0)

dibujar_mapa_niveles(f, rango=5.5, resolucion=150)

n_inicios = 8
tasa_aprendizaje = 0.05
n_iter = 80

mejor = None

for _ in range(n_inicios):
    p0 = [rng.uniform(-5, 5), rng.uniform(-5, 5)]
    p_final, f_final, trayectoria = ejecutar_descenso_gradiente(
        f, df, p0, tasa_aprendizaje=tasa_aprendizaje, n_iter=n_iter
    )

    if (mejor is None) or (f_final < mejor[0]):
        mejor = (f_final, p_final, trayectoria, p0)

f_mejor, p_mejor, tray_mejor, p0_mejor = mejor

plt.plot(p0_mejor[0], p0_mejor[1], "o", c="white")
plt.plot(tray_mejor[:, 0], tray_mejor[:, 1], "o", c="red", markersize=3)
plt.plot(p_mejor[0], p_mejor[1], "o", c="green")
plt.show()

print("Mejor inicio:", p0_mejor)
print("Mejor solución:", p_mejor.tolist(), "f(p)=", f_mejor)
```


4
2
0.84
0.63
0.42
0.21